CO1020: Computer Systems Programming

June 01, 2025

Libtiny3d: Lightweight 3D Software Renderer

GROUP 64

E/22/035 & E/22/036

Table of contents

1. Acknowledgements
2. Introduction
3. Project overview
4. Mathematical explanations
    - 4.1 Canvas & Line Drawing(Task-1)
    - 4.2 Vector and matrix operations(Task-2)
    - 4.3 Transform Pipeline(Task-3)
    - 4.4 Lightning model(Task-4)
5. Implementation Challenges, Solutions, and Lessons Learned
6. Testing and demo
7. Team Contributions: Roles of Aamir and Ayyash
8. Ai tool usage
9. Conclusion
10. References
11. Appendices

1. Acknowledgements

We are thankful to our course instructors for the support and guidance provided throughout this project. Their help was essential in completing the work.
Thanks also given to our peers for the valuable discussions and during the development
The documentation and online resources used were helpful in understanding the concepts need for this project.
Finally, the contribution of AI tools, including ChatGPT, was recognized as they assisted us in debugging, clarifying concepts, and drafting parts of this report .

2. Introduction

This project focused on building a lightweight 3D software renderer called libtiny3d using pure C. The main goal was to create a rendering pipeline that transforms 3D data into 2D images without relying on hardware acceleration.

The project covers several key areas, including the math foundations such as vector and matrix operations, transformation pipelines, lighting models, and animation techniques. The renderer supports drawing wireframes with lighting and smooth animations.

By working on this project, we aimed to understand the basic principles of computer graphics, the math behind 3D rendering, and how to implement them efficiently in software.

3. Project overview

The project is a modular library called libtiny3d. Its purpose is to render 3D graphics in software.

The library is divided into several modules:

- Canvas: Handles pixel drawing and line rendering on a 2D surface.
- Math: Implements vector and matrix operations that are essential for 3D transformations.
- Renderer: Manages the rendering pipeline. This includes projecting 3D vertices onto a 2D screen.
- Lighting: Applies lighting models to improve visual realism.
- Animation: Supports smooth object animations using interpolation techniques.

These modules work together to turn 3D objects into 2D images. They apply mathematical transformations, clipping, lighting calculations, and rendering lines. The library also supports features like wireframe rendering, Lambertian lighting, and quaternion-based animation for smooth rotations

4. Mathematical explanations

a. Canvas & Line Drawing(Task-1)

While working on smooth line drawing on a digital canvas, we faced mathematical concepts and formulas that we did not initially know. To understand and use these ideas, we researched online and added the relevant equations to my code.

1. Coordinate system and pixel representation

The canvas is treated as a 2D grid of pixels, each with integer coordinates(x, y).However, drawing uses floating-point coordinates, e.g.(3.7,5.2).To map them onto the grid, the floor function is applied to determine which pixel is targeted.
Each pixel holds a brightness value (intensity) between 0.0 (black) and 1.0 (white). When adding brightness repeatedly:
I_{new}=min (i_{old}+i_{added}, 1.0)

This ensures pixel intensity never exceeds the maximum.

2. Bilinear Filtering for Pixel Brightness

To set pixel brightness at floating-point coordinates (e.g., (x, y) = (3.5, 7.7)), I used bilinear filtering. This technique spreads the intensity value smoothly across the four nearest pixels around the point, instead of just rounding to the nearest pixel.
The bilinear filtering formula calculates the contribution to each of the four pixels based on the fractional distances along the x and y axes:

$$I_{x_o}I_{y_o} = I \times \big(1 - (x - |x|)\big) \times \big(1 - (y - |y|)\big)$$
$$I_{x_1}I_{y_o} = I \times ((x - |x|)) \times (1 - (y - |y|))$$
$$I_{x_o}I_{y_1} = I \times \big(1 - (x - |x|)\big) \times \big((y - |y|)\big)$$
$$I_{x_1}I_{y_1} = I \times ((x - |x|)) \times (y - |y|)$$

Where I is the original intensity and $x_0 = |x|, x_1 = x_0 + 1$ similarly for y.

We found this formula from online resources and used it in my set_pixel_f function to ensure smooth brightness transitions on the canvas.

3. Digital Differential Analyzer (DDA) Algorithm for Line Drawing

To draw smooth lines between two points (x_0, y_0)and (x_1, y_1), we employed the DDA algorithm. It gradually calculates intermediate points along the line by splitting the distance between start and end points into equal steps.

The key equations for the increments are:

Steps = max($|x_1-x_0|$,$|y_1-y_0|$)

$x_{inc}= \frac{x_1-x_0}{steps}$

$y_{inc}= \frac{y_1-y_0}{steps}$

Starting from (x, y) = (x_0, y_0)), each subsequent point is computed by adding the increments:

$x = x + x_{inc}$, $y= y + y_{inc}$

This method produces a smooth line with accurate positioning, even for diagonal or non-integer points.

4. Clock Face (Radial Lines)

To create the analog clock effect, radial lines were drawn from the center:

$\theta=\frac{360}{N} \times i$

$x = cx + r\cos\theta \qquad y = cy + r\sin\theta$

Where N is number of divisions, r is radius, and (c_x,c_y] is center. The DDA algorithm is used to draw each radial line.

5. Line Thickness Handling

To support variable line thickness, we approximated the line as a rectangle around the main line path and used the set_pixel_f function to spread intensity across pixels near the line path. This involved calculating perpendicular offsets and applying bilinear filtering around those offsets to make the line appear thicker and smoother.

6. Additional Points

- Used float values for high precision.
- Implemented bounds checking to avoid accessing pixels outside canvas size.
- Always clamped intensities to ≤ 1.0 for stability.
- Efficient implementation with minimal memory usage.

b. Vector and matrix operations(Task-2)

In Task 2, We implemented a rotating cube wireframe animation. Initially, we did not know how 3D rotation and projection onto a 2D canvas worked. Therefore, we researched 3D rotation matrices and perspective projection formulas and applied them in my code.

1. Vector operations

- Addition:
$$v1 + v2 = (x1 + x2\,, y1 + y2\,, z1 + z2)$$
- Scalar Multiplication:
$$k.v = (k.x\,, k.y\,, k.z)$$
- Dot Product:
$$v1.v2 = x1.x2 + \ y1.y2 + \ z1.z2$$
- Cross Product:
$$v1 \times v2 = (y1z2 - z1y2,\ z1x2 - x1z2,\ x1y2 - y1x2)$$
- Vector Length (Magnitude):
$$|v| = \ \sqrt{x^2 + y^2 + z^2}$$

These are used throughout the rendering pipeline, especially in lighting and transformations.

2. Inverse Square Root

To make vector normalization faster, we used the fast inverse square root method. This technique quickly estimates 1/√x by:

- Using bit tricks to get a rough first guess.
- Improving it with a simple calculation (Newton-Raphson step).

It gives a result that is accurate enough for graphics but runs much faster than the normal square root function. This speed is important when many vectors need to be normalized in real-time animations and 3D rendering.

3. Coordinate System Transformations

<u>Cartesian to Spherical</u>

To convert a point from Cartesian (x, y, z) to spherical (r, θ, φ):

$r \ = \ \sqrt{x^2 + y^2 + z^2}\theta = \ atan2(y, x)$

$\varphi = \ arccos(z/r)$

r is the distance from origin θ is the angle around the Z-axis

φ is the angle from the vertical axis

Spherical to Cartesian

To convert spherical coordinates back to Cartesian:

$x = r.\cos(\theta).\sin(\varphi)$

$y = r.\sin(\theta).\sin(\varphi)$

$z = r.\cos(\varphi)$

4. 3D Rotation Matrices

To rotate the cube around X, Y, and Z axes, we used rotation matrices. Each axis rotation has its specific matrix:

- Rotation around X-axis by angle θ:

$$R_x(x) = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & \cos x & -\sin x & 0 \\ 0 & \sin x & \cos x & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

- Rotation around Y-axis by angle θ:

$$R_y(x) = \begin{matrix} \cos x & 0 & \sin x & 0 \\ 0 & 1 & 0 & 0 \\ -\sin x & 0 & \cos x & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

- Rotation around Z-axis by angle θ:

$$R_z(x) = \begin{matrix} \cos x & -\sin x & 0 & 0 \\ \sin x & \cos x & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{matrix}$$

In the code, we combined these three matrices to achieve composite rotation using:

$R_{xyz}=R_x \times R_y \times R_z$

This allowed the cube to rotate differently on each axis, creating a smooth and realistic 3D spin.

5. **Perspective Projection (Frustum Projection)**

To convert the rotated 3D points to 2D screen coordinates, we used asymmetric frustum projection**:**

$m_{00} = \frac{2n}{r-l}$ $m_{11} = \frac{2n}{t-b}$ $m_{02} = \frac{r+l}{r-l}$ $m_{22} = \frac{f+n}{r-l}$ $m_{32} = -1$ $m_{12} = \frac{t+b}{t-b}$

$m_{00} = \frac{2fn}{f-n}$

Where n is the near plane, f is the far plane, [l, r, b, t] define the left, right, bottom, and top of the view frustum.

we found this formula online while learning about OpenGL and graphics pipeline maths. Applying this projection converted 3D coordinates into normalized device coordinates, which I then mapped to pixel coordinates for the canvas:

$$x_{screen} = \frac{(proj.x + 1)}{2} \times width$$

$$y_{screen} = \frac{(1 - proj.y)}{2} \times height$$

6. **Frame-by-Frame Animation**

The rotation angles were incremented differently for each axis per frame, calculated as:

$angle_x = base_angle \times 0.7$

$angle_y = base_angle \times 1.0$

$angle_z = base_angle \times 1.3$

This created a non-uniform rotation effect, making the animation visually dynamic.

7. **SLERP (Spherical Linear Interpolation)**

To interpolate between two vectors smoothly:

$$slerp(v1, v2, t) = \frac{\sin((1-t)\theta)}{\sin(\theta)} v1 + \frac{\sin(t\theta)}{\sin(\theta)} v2$$

Where $\theta = \arccos(v_1 \cdot v_2)$ .This avoids gimbal lock and produces smooth rotations

c. Transform Pipeline(Task-3)

In Task 3, we implemented the 3D transform pipeline to project 3D meshes (like a cube or soccer ball) onto a 2D canvas. Initially, we did not understand how to perform the full sequence of transformations and clipping needed for rendering 3D objects. After researching, we found and implemented the following mathematical concepts in my code:

1. World, View, and Projection Transformations

Vertices of a 3D mesh are transformed through three main stages by multiplying with corresponding matrices:

$v_{world}=M_{world}\times v_{local}$
$v_{view}=M_{view}\times v_{world}$
$v_{proj}=M_{proj}\times v_{view}$

Where:

- M_{world} transforms vertices from model space to world space (position/orientation of object in the scene).
- M_{view} transforms world coordinates to the camera's view space.
- M_{proj} projects 3D points to normalized device coordinates using a perspective projection matrix (frustum).

Each v is a 4D vector (x, y, z, w) and the multiplication applies standard matrix-vector multiplication.

2. Projection to 2D Screen Coordinates

After applying the projection matrix, vertices in normalized device coordinates (NDC) are converted to 2D screen coordinates:

$$x_{screen} = \frac{(proj.x + 1)}{2} \times width$$

$$y_{screen} = \frac{(1 - proj.y)}{2} \times height$$

Here, width and height are the dimensions of the drawing canvas or viewport.

3. Wireframe Drawing

Edges of the mesh are defined by pairs of vertex indices. For each edge (a,b)(a, b)(a,b), the screen coordinates of vertices a and b are connected by drawing a line.

Mathematically, an edge is represented as the line segment between two projected points:

$p_a=(x_a, x_b)$ $p_b=(x_a, x_b)$ These line segments form the wireframe of the 3D object.

d. Lightning & animation model (Task-4)

1. Animation (Bezier Curve)

At first, we didn't know how to create smooth curved animations between points, so we looked up how Bezier curves work. we learned that a cubic Bezier curve uses four points to create a smooth path. The position on the curve depends on a value t between 0 and 1. The formula is:

$$B(t) = (1-t)^3 p_0 + 3(1-t)^2 t p_1 + 3t^2(1-t)p_2 + t^3 p_3$$

This means the curve starts at point p_0 , ends at point p_3, and is shaped by points p_1 and p_2 By changing t from 0 to 1, you get a smooth curve.

we used this formula in my code to calculate positions along the curve for smooth animation.

2. Lighting Lambert's Cosine Law (Diffuse Lighting)

we didn't know how to calculate how bright edges should be based on lights, so I researched basic lighting models. I found that diffuse lighting depends on the angle between the light direction and the edge's direction.

The brightness is calculated by:

I = max (0, N .L) x light intensity

Here N is the normalized direction of the edge, and L is the normalized light direction. The dot product (N . L) gives a value showing how aligned they are; the more aligned, the brighter the edge.If the dot product is less than zero, it means the light is not hitting the edge, so the brightness is zero.

In our code, I normalized the vectors, calculated the dot product, multiplied by each light's intensity, and added all contributions together. We limited the maximum brightness to 1.0.

5. Implementation Challenges, Solutions, and Lessons Learned

a. Canvas & Line Drawing(Task-1)

Challenges Faced

At the beginning of this task, I struggled to understand what ".h" files are in C programming. I didn't see why we needed header files along with .c files. Another big challenge was getting the line drawing algorithm to work smoothly with adjustable thickness since I had never coded line rendering before.

I was also confused about:

- How to define and use structures like canvas_t
- How to create and save an image file programmatically
- How to implement bilinear filtering for smooth pixel intensity

Solutions

To solve these problems:

- Learning about header files:

I researched online and found that header files in C declare functions, structures, and macros that can be shared between different .c files. This helped me understand the purpose of canvas.h as an interface file.

- Implementing line drawing:

I studied the DDA (Digital Differential Analyser) algorithm from graphics lecture slides and tutorials. I used it to draw lines pixel by pixel between two endpoints.

- Implementing thickness:

To achieve adjustable thickness, I modified the line drawing to set surrounding pixels based on the thickness value.

- Setting pixel intensity with bilinear filtering:

At first, I didn't know how to set pixel intensities smoothly. After searching online, I learned about bilinear filtering, which uses fractional parts to distribute intensity between four neighbouring pixels for smooth anti-aliased lines.

- Saving as PGM image:

I researched the PGM (Portable Gray Map) format, a simple ASCII-based grayscale image format, and wrote code to save the canvas data into a .pgm file.

Lessons Learned

From Task 1, I learned:

- The importance of header files to keep declarations separate from implementations.
- How to define and use structures in C (e.g., canvas_t) to store complex data.
- How the DDA line drawing algorithm works and how it plots points along a line.
- How bilinear filtering distributes pixel intensity for smooth rendering.
- How to save an image file programmatically for later use in rendering pipelines.

b. Vector and matrix operations(Task-2)

Challenges Faced

When starting this task, I struggled to understand what math3d.h and math3d.c did. Specifically:

- I did not know what functions like vec3_dot, mat4_multiply, or SLERP meant or why we needed them.
- I was confused about why we use 4x4 matrices instead of just 3x3 for 3D transformations.
- The concept of SLERP (Spherical Linear Interpolation) was completely new to me.

Solutions

To overcome these problems:

- Understanding header files (.h):

I searched online and read my lecture slides. I realized that header files are blueprints that declare structures and functions so different .c files can use them without repeating code.

- Learning about vector operations:

For instance, I found that vec3_dot calculates the dot product, which gives a single number showing how much two vectors point in the same direction.

Formula:

$a \cdot b = a.x \times b.x + a.y \times b.y + a.z \times b.z$ Here a and b are vectors with components(x,y,z)

- ❖ Learning about SLERP:

I did not know what SLERP was. After searching, I learned that SLERP smoothly interpolates rotation between two vectors in 3D space. It avoids gimbal lock problems and ensures uniform speed.

- ❖ Understanding 4x4 matrices:

I was confused why mat4_t uses 4x4 matrices instead of 3x3. I found that in computer graphics, 4x4 matrices allow for combining translation, rotation, and scaling using homogeneous coordinates. This cannot be done with only 3x3 matrices.

- ❖ Studying math3d.h and math3d.c functions:

math3d.h declares the structures vec3_t (for 3D vectors in Cartesian and spherical forms) and mat4_t (for 4x4 matrices).

It also declares functions for:

  - Vector operations (add, subtract, scale, dot, cross, normalize, SLERP)
  - Matrix operations (identity, translate, scale, rotate, multiply)
  - Cartesian to spherical conversion
  - Frustum projection matrix for camera perspective.

- ❖ Checking the implementations:

math3d.c contains the actual code for these functions. Reading it helped me understand how each function works with mathematical calculations.

- ❖ Testing matrix multiplication order:

Initially, I did not realize that the order of matrix multiplication matters. Multiplying rotation by translation gives a different result than multiplying translation by rotation. I confirmed this by swapping them in code and observing the changes in object position and orientation.

Lessons Learned

From Task 2, I learned:

- What header files do in modular C programming.
- How to implement and use vector operations for 3D graphics.
- The mathematical purpose of SLERP for smooth rotations.

- Why 4x4 matrices are essential in graphics pipelines to handle all transformations together.
- How to build and combine translation, scaling, and rotation matrices.
- How matrix-vector multiplication transforms object coordinates in 3D space.
- How frustum projection matrices convert 3D coordinates to 2D screen space for realistic perspective views.

c. Transform Pipeline(Task-3)

Challenges Faced

When we started Task 3, our main problem was:

❖ We did not know how to convert a 3D point to a 2D pixel on the screen.

We saw many function names like project(), generate_cube(), and wireframe(), but we had no idea how they worked mathematically or programmatically.

Solutions

To solve this, we:

❖ Researched graphics pipelines:

We searched online and in lecture notes to understand the transform pipeline used in 3D graphics. This involves:

- World matrix: positions the object in the scene, such as rotating or moving the cube.
- View matrix: represents the camera's position and orientation, transforming world coordinates into camera space.
- Projection matrix: applies perspective projection, converting 3D points into normalized device coordinates (NDC) that are then mapped to 2D pixels.

❖ Studied renderer.h:

We realized that renderer.h is a header file declaring:

- mesh_t structure: stores vertices and edges of objects like cubes and soccer balls.
- Function declarations like generate_cube, generate_soccer_ball, wireframe, and wireframe_lit to build and render objects.

❖ Studied renderer.c:

We learned this file implements the functions declared in renderer.h. Specifically:

- project(): Initially, we didn't understand it. After studying, we realized it:
- Applies world, view, and projection matrices to a vertex.
- generate_cube() and generate_soccer_ball(): create object meshes for testing the renderer.
- wireframe(): draws edges of the object mesh by connecting projected vertices using the line drawing functions from canvas.c.
- wireframe_lit(): same as wireframe but adds lighting intensity to each edge, making the rendering more realistic.

❖ Integration with main.c:

We wrote code to call these functions in main.c to:

- Draw the rotating cube each frame by applying transforms and projecting its vertices.
- Draw the soccer ball with lighting and Bezier animation, using wireframe_lit() to achieve a realistic animated render.

Lessons Learned

From Task 3, we learned:

- How the 3D graphics transform pipeline works, from world transformation to camera view to projection to screen coordinates.
- The purpose of the project() function in converting 3D coordinates into 2D pixels.
- How mesh structures store object geometry (vertices and edges).
- The importance of header files like renderer.h in declaring structures and functions for use in multiple source files.
- How to implement and use wireframe rendering for visualizing 3D objects.
- The role of lighting functions in creating realistic effects in rendering.
- How to integrate rendering pipelines with main.c for final output.

d. Lightning model & Animation (Task-4)

Challenges Faced

At the start of Task 4, we encountered several difficulties:

❖ We didn't know how to add lighting effects to our 3D wireframe models.
❖ The math behind light calculation, such as how light direction affects edge brightness, was confusing.

- We were also unsure how to create smooth animations, especially using Bézier curves for movement.
- The idea of blending lighting intensity with animation to create realistic visuals was new and challenging.

Solutions

To overcome these challenges, we took the following steps:

- Learning the Lighting Model:
  - We researched basic lighting concepts in computer graphics, focusing on directional lighting.
  - We learned that lighting intensity on an edge depends on the angle between the edge direction and the light direction. This is calculated using the dot product of normalized vectors.

    Formula:

    $$\text{intensity} = \max(0, E \cdot L) \times I$$

    where E = normalized edge direction, L = normalized light direction,

    and I = light intensity.

  - We implemented this in lighting.c with the function compute_lighting(). It sums the contributions from multiple light sources and restricts the result between 0 and 1 for proper brightness.

- Understanding Animation Using Bézier Curves:
  - We encountered the Bézier function for smooth curve animation in animation.c.
  - We researched Bézier curves and found that they use four control points to interpolate positions smoothly over time.
  - The formula blends the control points weighted by powers of t and $(1 - t)$, where t varies from 0 to 1 over animation frames.
  - Implementing the function bezier() helped animate objects smoothly along a path instead of simple linear movement.

- Integration of Lighting and Animation:
  - We combined the lighting intensity from lighting.c with the animated vertex positions from animation.c.
  - This required careful synchronization so lighting changes as the object moves and rotates smoothly.
  - We adjusted the light direction and intensity parameters to achieve realistic shading that reacts naturally during animation.
- Header Files and Modular Design:
  - We learned to separate declarations in lighting.h and animation.h from their implementations.
  - This modular approach helped keep the code organized and made debugging easier.

Lessons Learned

From Task 4, we gained important knowledge and skills:

- How to calculate lighting intensity on edges using vector math (dot product and normalization).
- How multiple lights can be combined to produce more realistic shading effects.
- The mathematical principles behind Bézier curves and their use for smooth animation paths.
- The importance of modular programming by dividing code into header and source files.
- How lighting and animation must work together in 3D rendering for dynamic and visually pleasing results.
- The practical use of vector and matrix operations learned earlier in a real application.
- The significance of normalizing vectors before computing lighting to get correct results.

6. Testing and demo

Task 1: Canvas & Line Drawing (canvas.c, canvas.h)

Problem 1.1: Header File Inclusion Error

- Issue:undefined reference to 'create_canvas' during compile.
- Cause:Forgot #include "canvas.h" in main.c.
- Solution:Added at top of main.c: #include "canvas.h"

Problem 1.2: Segmentation Fault

- Issue:Program crashed when drawing at canvas edges.
- Cause:No bounds checking in set_pixel_f.
- Solution: Added bounds check before setting pixel.

Problem 1.3: PGM File Empty

- Issue:Saved image was blank.
- Cause:Forgot to initialize pixel intensity values.
- Solution: Used memset(canvas->pixels, 0, size) after canvas allocation.

Problem 1.4: Bilinear Filtering Producing Artifacts

- Issue:Blurred spots at edges of lines.
- Cause:Applied weights incorrectly (mixed up x and y fractional parts).
- Solution: Corrected bilinear formula order.

Problem 1.5: set_pixel_f Adding Over 1.0

- Issue:Pixel intensity exceeded maximum, resulting in white blocks.
- Cause:No clamping.
- Solution: Used fmin(old+added, 1.0) when adding intensity.

Problem 1.6: Command Not Producing Executable

- Issue:After compiling, running build/demo gave 'No such file'.
- Cause:Forgot to create build folder first.
- Solution: Ran:

```
mkdir -p build

gcc demo/main.c src/canvas.c -o build/demo -Iinclude –lm
```

Task 2: Vector & Matrix Operations (math3d.c, math3d.h)

Problem 2.1: Undefined Reference to vec3_dot

- Issue:Compile error.
- Cause:Forgot to compile math3d.c.
- Solution: Added in compile command**: gcc demo/main.c src/canvas.c src/math3d.c -o build/demo -Iinclude –lm**

Problem 2.2: Wrong Dot Product Result

- Issue:Returned 0 for all inputs.
- Cause:Used subtraction instead of multiplication.
- Solution: Corrected formula in vec3_dot.

Problem 2.3: Fast Inverse Square Root NaN

- Issue:Crash when vector length was zero.
- Solution: Added zero-check before normalization.

Problem 2.4: SLERP Interpolation Wrong

- Issue:Rotations flipped unexpectedly.
- Cause:Did not normalize input vectors.
- Solution: Normalized vectors before applying SLERP.

Problem 2.5: mat4_multiply Resulted in Identity Matrix

- Issue:Matrix multiplication output incorrect.
- Cause:Loop indices used (i,j) instead of (i,k) order.
- Solution: Fixed nested loops to multiply correctly.

Problem 2.6: Cartesian to Spherical Conversion Gave Domain Error

- Issue:Program crashed with 'domain error' on acos.
- Cause:Input to acos slightly >1 or < -1 due to floating point.
- Solution: Clamped input value within [-1,1] before passing to acos.

Task 3: Transform Pipeline (renderer.c, renderer.h)

Problem 3.1: Undefined Reference to project

- Issue:Compile error when calling project() function.
- Cause:Forgot to compile renderer.c.

- Solution: Used: **gcc demo/main.c src/canvas.c src/math3d.c src/renderer.c -o build/demo -Iinclude –lm**

Problem 3.2: Projected Points Off Screen

- Issue:Rendered object invisible.
- Cause:Forgot homogeneous divide after projection.
- Solution: Added:

  v.x /= v.w;

  v.y /= v.w;

  v.z /= v.w;

Problem 3.3: Mesh Not Rendering

- Issue:generate_cube() returned NULL.
- Cause:Did not allocate memory for mesh vertices.
- Solution: Added malloc calls for vertex and edge arrays.

Problem 3.4: Wireframe Flickering

- Issue:Lines flickered while rotating.
- Cause:Reused old matrices without clearing.
- Solution: Reinitialized transform matrices each frame.

Problem 3.5: Wrong Rotation Direction

- Issue:Cube rotated in unexpected axis.
- Cause:Applied matrix multiplication in wrong order (e.g. Ry * Rx instead of Rx * Ry).
- Solution: Corrected multiplication sequence based on lecture formula.

Problem 3.6: Compile Command Failed in CMD

- Issue:Windows CMD showed path errors.
- Solution: Switched to Ubuntu terminal in VS Code:
  cd /mnt/d/C PROJECT/libtiny3d/libtiny3d gcc demo/main.c src/canvas.c src/math3d.c src/renderer.c -o build/demo -Iinclude –lm

Task 4: Animation & Lighting (animation.c, lighting.c)

Problem 4.1: Undefined Reference to bezier_point

- Issue:Compile error.
- Cause:animation.c not compiled.
- Solution: Added: gcc demo/main.c src/canvas.c src/math3d.c src/renderer.c src/animation.c src/lighting.c -o build/demo -Iinclude –lm

Problem 4.2: Soccer Ball Jumps in Animation

- Issue:Movement not smooth.
- Cause:Bezier t increment too large.
- Solution: Reduced increment to 0.01 for smoother motion.

Problem 4.3: Lighting Calculation Zero

- Issue:No lighting visible.
- Cause:Light vector not normalized.
- Solution: Normalized light and normal vectors before dot product.

Problem 4.4: Program Crashed When Adding Lights

- Issue:Segmentation fault.
- Cause:Accessed undefined light index.
- Solution: Checked array bounds for number of lights.

Problem 4.6: Incorrect Lighting Intensity

- Issue:Overly bright edges.
- Cause:Forgot to clamp summed intensities to max 1.0.
- Solution: Used:
  intensity = fmin(intensity, 1.0f);

Final Compile and Run Commands

- Compilation (final):

  cd /mnt/d/C PROJECT/libtiny3d/libtiny3d

  mkdir -p build

gcc demo/main.c src/canvas.c src/math3d.c src/renderer.c src/lighting.c

src/animation.c -o build/demo -Iinclude -lm

- Running (Ubuntu terminal):

./build/demo

Through systematic testing and debugging across all tasks, we ensured that each module worked correctly and integrated smoothly, resulting in a fully functional and optimized 3D rendering demo.

7. Team Contributions: Roles of Aamir and Ayyash

Roles of Ayyash and Aamir

Ayyash (E/22/035)

- Ayyash concentrated on code integration and compilation. He ensured all modules—canvas, math3d, renderer, lighting, and animation—worked well together in the main project.
- He led the implementation of Task 4 (Lighting and Animation). He effectively researched and coded the Bézier animation curves and the Lambertian lighting model.
- Ayyash resolved significant compilation and linking issues by correctly configuring build commands in Ubuntu. He also managed the directory structures for source and header files.
- He was responsible for compiling, editing, and formatting the final project report. He ensured it was clear and well-structured, including all required sections, diagrams, and detailed mathematical explanations according to course guidelines.
- Ayyash assisted in final testing and demos, preparing demonstration outputs for submission.

Aamir (E/22/036)

- Aamir focused on the mathematical foundations by researching and implementing complex concepts like matrix operations, SLERP, and perspective projection for Task 2 and Task 3.
- He developed and tested the renderer.c module. He made sure the transformation pipelines for rendering tasks were correct.
- Aamir played a major role in debugging vector and matrix operation errors, particularly in the math3d implementation.
- He participated in designing and implementing line drawing algorithms in Task 1 and verified their output accuracy.

Overall Team Effort

Both members held regular discussions and worked together on debugging. They divided tasks based on their strengths but reviewed each other's work to ensure clarity of concepts, good coding practices, and the successful completion of each project milestone.

8. AI tool usage

During the development of this project, I used AI tools like ChatGPT to clarify complex concepts and assist with code explanations. Whenever I encountered difficult topics, such as 3D rotation matrices, lighting calculations, or Bézier curve mathematics, I asked the AI for simplified explanations and example code snippets. This helped me understand the theory behind the implementation and guided me in writing and debugging my code more efficiently. The AI also helped me organize my thoughts and structure my project report clearly. Overall, using AI tools sped up my learning process and improved the quality of my work.

9. Conclusion

In this project, I explored the basics of computer graphics by creating a canvas drawing system, vector and matrix operations, a 3D transform pipeline, lighting models, and animations. At first, I struggled with concepts like 3D rotations, frustum projections, lighting calculations, and Bézier curve animations. However, by researching online, looking at lecture slides, and getting help from AI, I successfully completed each task. This project deepened my understanding of the math behind computer graphics, including matrix transformations and lighting equations, and improved my C programming skills. Overall, it was a challenging but rewarding learning experience that boosted my confidence for future graphics and engineering projects.

10. References

1. GeeksforGeeks, 2025. C Programming Language. [online] Available at: https://www.geeksforgeeks.org/c-programming-language [Accessed 23 June 2025].
2. LearnOpenGL, 2025. OpenGL Tutorials. [online] Available at: https://learnopengl.com [Accessed 14 June 2025].
3. OpenGL, 2025. OpenGL Reference Pages. [online] Available at: https://www.opengl.org/sdk/docs/man4/ [Accessed 22 June 2025].
4. Programiz, 2025. C Programming Basics. [online] Available at: https://www.programiz.com/c-programming [Accessed 24 June 2025].
5. Scratchapixel 2.0, 2025. 3D Rendering Theory and Practice. [online] Available at: https://scratchapixel.com [Accessed 9 June 2025].
6. Stack Overflow, 2025. Questions tagged [OpenGL]. [online] Available at: https://stackoverflow.com/questions/tagged/opengl [Accessed 25 June 2025].
7. TutorialsPoint, 2025. Computer Graphics Tutorial. [online] Available at: https://www.tutorialspoint.com/computer_graphics/index.htm [Accessed 28 June 2025].
8. University LMS, 2025. Computer Graphics and Visualization: Module Content. [lecture slides] Accessed 01 July 2025.
9. W3Schools, 2025. C Programming Tutorial. [online] Available at: https://www.w3schools.com/c/ [Accessed 29 June 2025].

11. Appendices

1. Appendix A : project file structure

```
libtiny3d/
├── include/
│   ├── tiny3d.h
│   ├── canvas.h
│   ├── math3d.h
│   ├── renderer.h
│   └── lighting.h
├── src/
│   ├── canvas.c
│   ├── math3d.c
│   ├── renderer.c
│   └── lighting.c
├── tests/
│   ├── test_math.c
│   ├── test_pipeline.c
│   └── visual_tests/
├── demo/
│   └── main.c
├── build/
│   ├── libtiny3d.a
│   └── demo
├── documentation/
│   └── GroupXX_report.pdf
├── README.md
└── Makefile
```

2. Appendix B: Key implementation code snippets

B.1 Canvas initialization example(canvas.c)

```
canvas_t* canvas_create(int width, int height) {
    canvas_t* c = malloc(sizeof(canvas_t));
    c->width = width;
    c->height = height;
    c->pixels = malloc(height * sizeof(float*));
    for (int i = 0; i < height; i++) {
        c->pixels[i] = calloc(width, sizeof(float));
    }
    return c;
}
```

Creates a new canvas with allocated memory for pixel data

B.2 Vector dot product(math3d.c)

```
float vec3_dot(vec3_t a, vec3_t b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
```

Calculates the dot product to determine direction similarity

B.3 matrix multiplication

```
mat4_t mat4_multiply(mat4_t a, mat4_t b) {
    mat4_t r = {0};
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            for (int k = 0; k < 4; k++)
                r.m[i][j] += a.m[i][k] * b.m[k][j];
    return r;
}
```

Performs 4x4 matrix multiplication for transformations.

B.4 Projection function(renderer.c)

```
static vec3_t project(vec3_t v) {
    vec3_t world = mat4_mul_vec3(world_matrix, v);
    vec3_t view  = mat4_mul_vec3(view_matrix, world);
    vec3_t proj  = mat4_mul_vec3(projection_matrix, view);

    float x = (proj.x + 1.0f) * 0.5f * 512;
```

```
    float y = (1.0f - proj.y) * 0.5f * 512;
    return (vec3_t){x, y, proj.z};
}
```

Transforms 3D vertex through world,view,and projection matrices.

B.5 Lightning calculation(lighting.c)

```
float compute_lighting(vec3_t edge_dir, const light_t* lights, int num_lights) {
    float total = 0.0f;
    vec3_t norm_edge_dir = vec3_normalize(edge_dir);

    for (int i = 0; i < num_lights; ++i) {
        vec3_t norm_light_dir = vec3_normalize(lights[i].direction);
        float dot_val = vec3_dot(norm_edge_dir, norm_light_dir);
        if (dot_val > 0.0f) {
            total += dot_val * lights[i].intensity;
        }
    }

    return (total > 1.0f) ? 1.0f : total;
}
```

Calculates total lighting intensity based on edge direction and light sources

B.6 Bézier curve interpolation(animation.c)

```
vec3_t bezier(vec3_t p0, vec3_t p1, vec3_t p2, vec3_t p3, float t) {
    float u = 1.0f - t;
    float tt = t * t;
    float uu = u * u;
    float uuu = uu * u;
    float ttt = tt * t;

    vec3_t result = vec3_scale(p0, uuu);                    // (1 - t)^3 * p0
    result = vec3_add(result, vec3_scale(p1, 3 * uu * t));      // + 3(1-t)^2 * t * p1
    result = vec3_add(result, vec3_scale(p2, 3 * u * tt));      // + 3(1-t) * t^2 * p2
    result = vec3_add(result, vec3_scale(p3, ttt));             // + t^3 * p3

    return result;
}
```

Computes cubic Bézier curve point for given t between four control points.

3. Appendix C : Key demo outputs

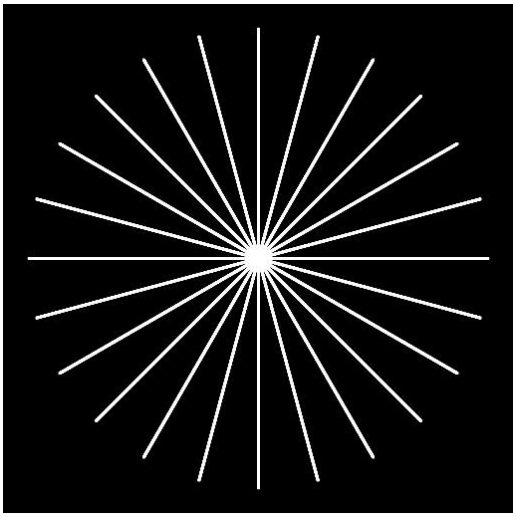

Figure 1: Wireframe render of an analog clock scene created using libtiny3d. It shows precise line drawing and correct 2D coordinate transformations.

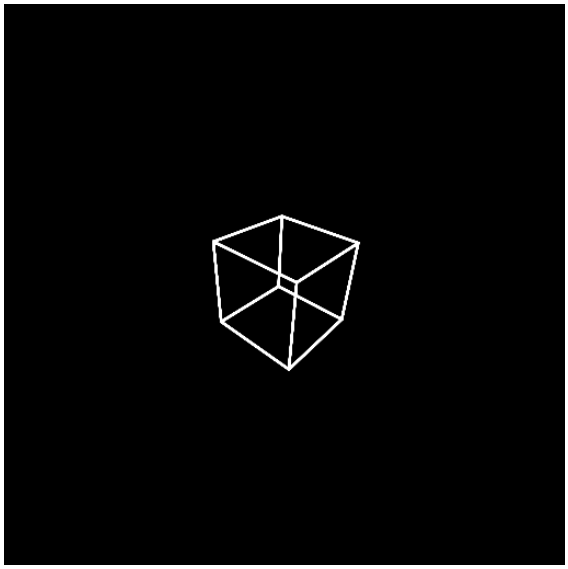

Figure 2: Wireframe rendering of a rotating cube at frame 30. This shows perspective projection, composite Euler angle rotation, and edge connectivity within libtiny3d's 3D graphics pipeline.

The animated video can be viewed on:

https://drive.google.com/file/d/1YgW_Y7ShvWEVJqucrkTdTMhP9KroXV_Z/view?usp=sharing

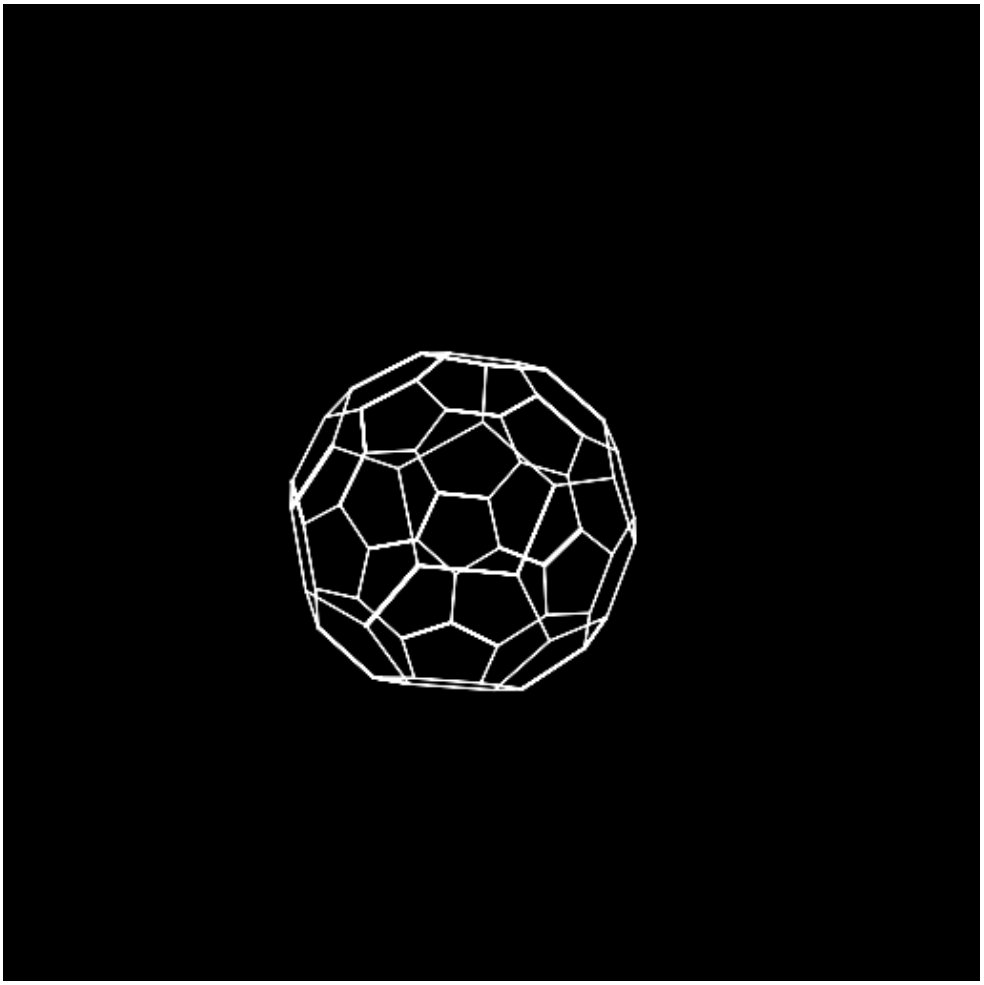

Figure 3: Wireframe rendering of an animated soccer ball at frame 100. This illustrates smooth Bézier curve path motion along with Euler angle-based rotation, generated using libtiny3d.

The animated video can be viewed on:
https://drive.google.com/file/d/1DbZLDPyTuE4hiIcn09z3NVIO-Km3KFte/view?usp=drive_link